UNIVERSITÀ
DEGLI STUDI
DI PADOVA

DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

INFORMATION ENGINEERING DEPARTMENT

MASTER'S DEGREE IN COMPUTER ENGINEERING

Card Counting System using YOLO11

Professor
Stefano Ghidoni

Students
Marco Annunziata, 2160851
Hermann Serain, 2158294
Sveva Turola, 2160852

ACADEMIC YEAR 2025-2026

Contents

1 Introduction **1**

2 Configuration **1**

3 Implementation **2**

4 Datasets and Experiments **2**

5 Results **2**

6 Contributions **2**

References **2**

1 Introduction

Blackjack is a popular casino game where players compete individually against the dealer (the house). In this game, the house generally maintains a slight advantage over the players and for this reason players often employ strategies, such as card counting, to mitigate this disadvantage.
Card counting involves monitoring the sequence of high and low-value cards already dealt to estimate the composition of the remaining deck. By maintaining a "running count," a player can identify advantageous moments to adjust their betting patterns.

This project proposes the development of an automated card counting detection system.
The primary goal is to build a video analysis pipeline capable of interpreting the flow of cards dealt during a game. Through this process, the system is able to monitor the game status and identify situations favorable to the players, alerting to focus on players' decisions and betting variations.

The project focuses on the implementation of the Hi-Lo counting method, where each card is assigned a specific value to update the total count:

- Low-value cards (2-6): assigned a value of +1;
- Neutral cards (7-9): assigned a value of 0;
- High-value cards (10-Ace): assigned a value of -1.

To facilitate intuitive monitoring, the system overlays color-coded bounding boxes around detected cards (Green for +1, Blue for 0, and Red for -1), allowing for real-time interpretation of the counting logic.

This project aims to show how computer vision can automate blackjack tracking. By using real-time detection and Hi-Lo logic, the system monitors the deck and visualizes the count, demonstrating the potential for automated monitoring in gaming environments.

2 Configuration

The project is organized to ensure a separation between source code, documentation, data and results. In particular, the directory structure is organized as follows:

- `build/`: contains all the files generated during the compilation process;
- `data/model/`: contains the YOLO11 models in ONNX format and the `labels.txt` file with all card symbols;
- `data/test/`: stores the images and videos used for testing, along with their manual ground truth annotations;
- `doxygen_docs/` and `doxygen.txt`: used to generate automatically a documentation of the source code;
- `external/`: includes the ONNX Runtime library and installation scripts required for model execution;
- `include/`: contains the C++ header files that define the project's classes and functions;
- `output/`: represents the folder where the final results are saved. If the input is a video, it contains a folder with all the relevant frames and the output video, while if the input is a image, it stores the image with the resulting Hi-Lo system;
- `scripts/`: contains Python scripts used for training and dataset preparation;
- `src/`: contains the C++ source files, which include the main file `finalProject.cpp`, specific files for each group member and a `shared.cpp` file for common utilities;
- `CMakeLists.txt`: configuration file used to manage the build process for all C++ components;
- `README.md`: provides an overview of the project and instructions for setup and execution.

3 Implementation

4 Datasets and Experiments

5 Results

6 Contributions